ListingBug — Developer Reference

Last Updated: March 29, 2026
Live Site: https://thelistingbug.com
GitHub: https://github.com/thexomouth/ListingBug

---

1. What Is ListingBug

ListingBug is a commercial SaaS platform for real-estate service providers (agents, brokers, investors, lenders) to monitor new property listings and harvest listing agent contact info. Users search the RentCast listing database, save and export results to CSV, and automate recurring searches that push data directly into their CRM, email marketing, or automation tools.

Target user: A real estate professional who wants to identify new listings in a market and immediately reach the listing agent — at scale, on a schedule, without manual work.

Core value loop:

1. Search listings by location + filters
2. Save/export the results (listing + agent contact info)
3. Set up an automation to repeat that search on a schedule
4. Connect a CRM/email tool to receive those results automatically

---

2. Tech Stack

Layer	Tool	Notes
Frontend	React 18 + TypeScript	Vite build, `"type": "module"`
Styling	Tailwind CSS v4	`@tailwindcss/vite` plugin
Component library	Radix UI primitives	shadcn-style composition
Routing	React Router DOM v6	SPA — all routes in `src/App.tsx`
Backend / DB	Supabase	PostgreSQL + Auth + Edge Functions

Listing data	RentCast API	Called only via edge function — account is in overages
Payments	Stripe	Checkout + Webhooks + Customer Portal
Deployment	Vercel	Auto-deploys on push to `main`
Version control	GitHub (`thexomouth/ListingBug`)	
Domain	Namecheap → Vercel	`thelistingbug.com` + `www.`

Key libraries: `lucide-react` (icons), `recharts` (charts), `sonner` + `react-toastify` (toasts), `react-hook-form`, `react-day-picker`, `next-themes`, `motion`

Dev environment: Windows 10, Node v24, VS Code at `C:\Users\User\Downloads\ListingBug FIGMA MVP`. Build: `npm run build` (Vite → `build/`).

---

3. Architecture

Frontend

Single-page application. All routing lives in `src/App.tsx`. Components are lazy-loaded (Suspense) except for auth pages and Dashboard, which load immediately to minimize time-to-interactive.

Critical pattern: `SubscriptionGate` is inlined as a function declaration in `App.tsx` after all imports — it must never be a separate file or top-level import due to a known TDZ (temporal dead zone) crash caused by circular Supabase imports.

Backend (Supabase)

- Auth: Email/password + Google OAuth. On signup, a database trigger auto-creates a row in the `users` table.
- Database: PostgreSQL with RLS enabled on all tables.
- Edge Functions: Deno-based serverless functions deployed to Supabase. The frontend calls these for anything requiring secrets (RentCast key, Stripe key, integration API keys).

Data Flow (Search)

1. User fills search form → clicks Search
2. Frontend calls `search-listings` edge function with filters + user JWT

3. Edge function: validates plan/trial limits → calls RentCast API → upserts results to `listings` table → inserts `search_runs` row
4. Frontend reads results from the edge function response, displays in table
5. User can save individual listings (writes to `saved_listings`) or save the search (writes to `searches`)

Data Flow (Automation)

1. User creates automation (name + search criteria + schedule + integration destination)
2. Row inserted into `automations` table (`active: true`)
3. Scheduled trigger (external cron or Supabase cron) calls `run-automation` edge function
4. Edge function: runs the search → dispatches results to the configured integration (Mailchimp, HubSpot, Google Sheets, etc.) via the corresponding `send-to-*` edge function
5. Result written to `automation_runs` table

---

4. Supabase Database Tables

Table	Purpose
`users`	Auth + plan + billing. Key cols: `plan`, `plan_status`, `trial_ends_at`, `stripe_customer_id`, `stripe_subscription_id`, `trial_abuse_flag`
`searches`	Saved search criteria per user
`saved_listings`	Bookmarked listings per user
`search_runs`	History of each search execution + result metadata
`automations`	Automation config: criteria, schedule, integration destination. `active` (bool) controls on/off — never use `status`
`automation_runs`	Execution history for automations
`integration_connections`	OAuth tokens + API keys per user per service
`usage_tracking`	Monthly listing fetch counts per user
`api_keys`	User-generated API keys for external access

`listings`	RentCast listing cache (all fields)
`property_records`	Full public record data
`property_valuations`	AVM results per user request
`property_history`	Listing/transaction history per user request
`tax_assessments`	Tax assessor data
`market_statistics`	Zip-level market data cache
`signup_fingerprints`	Trial abuse detection
`notifications`	In-app notification queue

Critical DB conventions:

- Profile updates: always use `.update()` — never `.upsert()` on `users` table (causes ghost row bugs)
- Google integration ID is stored as `'google'` (not `'sheets'`)
- Plan value `'professional'` maps to display label `'pro'`

---

5. Edge Functions

All deployed to Supabase project `ynqmisrlahjberhmlviz`.

Function	Version	Purpose
`search-listings`	v34	Core search: auth check, plan limit enforcement, RentCast call, usage tracking
`run-automation`	v12	Execute an automation: run search → dispatch to integration

`send-to-mailchimp`	v3	Push results to Mailchimp audience
`send-to-sheets`	v3	Push results to Google Sheets
`send-to-hubspot`	v3	Push results to HubSpot CRM
`send-to-sendgrid`	v4	Send via SendGrid
`send-to-twilio`	v4	SMS via Twilio
`webhook-push`	v3	POST results to any webhook URL
`get-integration-options`	v1	Loads dynamic config (Mailchimp audiences, SendGrid lists)
`create-checkout-session`	v3	Stripe Checkout
`stripe-webhook`	v3	Subscription lifecycle → updates `users` table
`stripe-portal`	v3	Redirect to Stripe Customer Portal
`delete-user`	v1	Full account deletion
`update-password`	—	Password change (uses native Supabase auth, not a custom edge fn)

RentCast warning: The account is in overages. Never call `search-listings` without explicit user intent.

---

6. Plans & Billing

Plan	Price	Listings/mo	Automations	Trial

Trial	$0	1,000	3	14 days
Starter	$19/mo	4,000	1	—
Professional	$49/mo	10,000	3	—
Enterprise	TBD (contact)	Unlimited	Unlimited	—

Stripe Price IDs:

- Starter: `price_1TDod6A3dmARSc7xs4IGkHwB`
- Professional: `price_1TDog0A3dmARSc7xuoR2gRUh`

`plan_status` mirrors Stripe subscription status: `active`, `trialing`, `canceled`, `past_due`, `unpaid`, `incomplete_expired`. The `SubscriptionGate` overlay in `App.tsx` blocks UI access when status is expired/past-due.

---

7. Integrations

9 confirmed integrations at launch. All configuration stored in `integration_connections` table.

CRM

Integration	Auth Method	Status
HubSpot	OAuth	Live
Salesforce	OAuth	Live
Zoho CRM	OAuth	Live

Email Marketing

Integration	Auth Method	Status
Mailchimp	OAuth	Live — audience dropdown dynamic
SendGrid	API key	Live

Constant Contact	OAuth	Live

Automation / Workflows

Integration	Auth Method	Status
Zapier	Webhook URL	Live
Make.com	Webhook URL	Live
n8n	Webhook URL	Live
Webhook (generic)	URL + optional Auth header	Live

Google Sheets is connected (OAuth) but the OAuth client credentials in Supabase secrets need to be updated to the "Listingbug sheets" client (not the sign-in client) — see Known Issues.

---

8. Site Map & Per-Page Breakdown

Public (unauthenticated) Pages

`/` — Home Page (`HomePage.tsx`)

Marketing landing page. Hero, feature overview, pricing teaser, testimonials, CTA to sign up. No backend calls.

`/how-it-works` — How It Works (`HowItWorksPage.tsx`)

Step-by-step product walkthrough. Static marketing content.

`/data-sets` — Data Sets (`DataSetsPage.tsx`)

Explains the RentCast listing data available. Static content.

`/use-cases` — Use Cases (`UseCasesPage.tsx`)

Use-case examples by role (agent, lender, investor, etc.). Static.

`/integrations` — Integrations Marketing (`IntegrationsMarketingPage.tsx`)

Marketing overview of all 9 integrations. Links to `/integrations/guide`.

`/automation` — Automation Overview (`AutomationPage.tsx`)

Marketing page explaining the automation feature. Static.

`/sample-report` — Sample Report (`SampleReportPage.tsx`)

Demonstrates what a search result looks like with fake data. No auth required.

`/login` — **Login** (`LoginPage.tsx`)

Email + password login, Google OAuth. Uses Supabase Auth. On success → `/dashboard`.

`/signup` — **Sign Up** (`SignUpPage.tsx`)

Email + password registration, Google OAuth. Creates Supabase auth user → trigger creates `users` row with `plan: 'trial'`.

`/forgot-password` — **Forgot Password** (`ForgotPasswordPage.tsx`)

Sends Supabase password reset email.

`/reset-password` — **Reset Password** (`ResetPasswordPage.tsx`)

Handles Supabase reset link. Updates password via `supabase.auth.updateUser`.

`/privacy` — **Privacy Policy** (`PrivacyPolicyPage.tsx`)

Static legal page.

`/terms` — **Terms of Service** (`TermsOfServicePage.tsx`)

Static legal page.

`/about`, `/careers`, `/contact`, `/blog`, `/changelog` — **Info Pages**

Static content pages. No backend calls.

`/help` — **Help Center** (`HelpCenterPage.tsx`)

FAQ and support articles. Static.

`/support` — **Contact Support** (`ContactSupportPage.tsx`)

Support form. Currently static (form submission not wired to backend).

Authenticated Pages

`/welcome` — **Welcome** (`WelcomePage.tsx`)

First-time post-signup landing. Intro to the product + CTA to start searching.

`/quick-start` — **Quick Start Guide** (`QuickStartGuidePage.tsx`)

Onboarding walkthrough. Linked from Welcome page.

`/dashboard` — **Dashboard** (`Dashboard.tsx`)

Primary landing after login. Shows:

- Usage stats (listings used this month / limit)
- Recent searches
- Saved listings count
- Active automations

- Recent activity feed
  All data read live from Supabase.

`/listings` — Search Listings (`SearchListings.tsx`)

Core feature. Large component (~108KB — surgical edits only). Search form with location, property type, beds, baths, price range, days on market, and more. Submits to `search-listings` edge function. Shows results inline with sortable columns, save button per row, export to CSV. History tab shows past searches (reads from `search_runs`).

Known issue: History tab shows location (not search name) for older runs — only new runs save the search name.

`/listings/results` — Search Results (`SearchResultsPage.tsx`)

Loads a specific saved search run from `search_runs` table and displays results. Used when navigating from dashboard history.

`/automations` — Automations Management (`AutomationsManagementPage.tsx`)

Two tabs:

- My Automations: Lists all automations (`active` bool toggle, run now, edit, delete, duplicate)
- History: Lists `automation_runs` records

Uses `AutomationPage`, `ViewEditAutomationDrawer`, `CreateAutomationModal`.

`/automations/detail/:id` — Automation Detail (`AutomationDetailPage.tsx`)

Full detail view for a single automation. Edit name, criteria, schedule, integration, active status.

`/automations/results` — Automation Run Results (`AutomationRunPage.tsx`)

Shows listings returned from a specific automation run. Reads from `automation_runs` + joins to listing data.

`/saved` — Saved Listings (`SavedListingsPage.tsx`)

All listings bookmarked by the user. Reads from `saved_listings`. Columns customizable. Export to CSV. Can delete saved listings.

`/account` — Account (`AccountPage.tsx`)

Five tabs:

- Profile: Name, company, role, email (read-only), password change
- Usage: Monthly listing usage chart vs. plan limit
- Billing: Current plan, subscription status, upgrade/downgrade (Stripe portal), billing history
- Integrations: Connect/disconnect integrations, view connection status, configure settings
- Compliance: Consent ledger / data provenance records

Integrations tab uses `IntegrationConnectionModal` for connect flow, `IntegrationDetailsPanel` for viewing connected integration details, `IntegrationManagementModal` for managing settings.

`/billing` — **Billing Page** (`BillingPage.tsx`)

Dedicated billing view (also accessible from Account → Billing tab). Shows current plan, payment method, invoice history from Stripe.

`/agents` — **Agents** (`AgentsPage.tsx`)

AI agents feature page — currently a placeholder/coming soon UI.

Utility / Internal Pages

`/api-docs` — **API Documentation** (`APIDocumentationPage.tsx`)

Developer docs for the ListingBug public API. Currently static placeholder.

`/api-setup` — **API Setup** (`APISetupPage.tsx`)

UI for generating and managing API keys (reads/writes `api_keys` table).

`/request-integration` — **Request Integration** (`RequestIntegrationPage.tsx`)

Form to request a new integration. Currently static (not wired to a backend).

`/integrations/guide` — **Integration Setup Guide** (`IntegrationSetupGuidePage.tsx`)

Step-by-step guides for setting up each integration. Static content.

`/design-system` — **Design System Demo** (`DesignSystemDemo.tsx`)

Internal component gallery. Not linked from nav. For dev reference only.

`/microcopy-pack`, `/consent-panel`, `/consent-modal` — **Consent Demos**

Internal demo pages for consent/compliance UI components. Not linked from nav.

---

9. Authentication Flow

1. User submits email + password (or Google OAuth) on Login/Signup
2. Supabase Auth issues JWT
3. App stores session in localStorage (Supabase handles this)
4. `supabase.auth.onAuthStateChange` in `App.tsx` detects session → sets `isLoggedIn = true`
5. App fetches user profile from `users` table using `supabase.auth.getUser()` then `.from('users').select()`
6. All edge function calls include the JWT in the `Authorization` header (Supabase client handles this automatically)
7. On logout: `supabase.auth.signOut()` → session cleared → redirect to home

Subscription gating: After profile load, if `plan_status` is expired/past-due/canceled, `SubscriptionGate` overlay renders over entire app, blocking navigation and prompting upgrade.

---

10. Known Bugs & Issues

Critical (requires action)

Google Sheets OAuth not persisting
The `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` in Supabase Edge Function Secrets point to the sign-in OAuth client, not the "Listingbug sheets" OAuth client. Sheets connections fail to persist after token refresh.
Fix: Supabase Dashboard → Project Settings → Edge Function Secrets → update `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` to the "Listingbug sheets" client values.

Google OAuth Consent Screen unverified
Limits sign-up/sign-in to 100 test users until Google verifies the consent screen.
Fix: Submit for verification at Google Cloud Console (requires live privacy policy URL).

Medium Priority

ActivateAutomationModal — per-integration config incomplete
Only Mailchimp has a dynamic config UI (audience dropdown + tags). These integrations still need dynamic config fields:

- Google Sheets: spreadsheet ID input + sheet name
- HubSpot: pipeline dropdown + object type
- SendGrid: list dropdown
- Twilio: from-number E.164 input
- Airtable: base ID + table ID

Search history shows location instead of name
Older `search_runs` records show the location string in the history tab, not the search name, because the name field wasn't stored in early runs. New runs are fine.

Usage cap UI enforcement
When a trial user approaches their 1,000 listing cap, there's no proactive toast/modal warning in the UI. They only find out when the edge function rejects the search.

Billing history
Invoice history on BillingPage requires real Stripe data to verify display logic.

API key generator
May have a display bug — needs QA.

Low Priority / Post-Launch

Facebook OAuth — hidden pending DBA registration
Apple OAuth — pending $99 Apple Developer account
Light mode — dark mode only currently; light mode restoration guide was documented but not implemented
Support form — `/support` form submission not wired to any backend

---

11. Planned Improvements

Near-term (QA / stabilization)

- QA all 8 core flows end-to-end after recent fixes (search, auth, automations, integrations, billing)
- Fix Google Sheets OAuth credential mismatch
- Submit Google OAuth consent screen for verification
- Add usage cap warning toast/modal before hitting the limit
- Complete per-integration config UI in ActivateAutomationModal

Feature Roadmap

- Agents page — AI agent feature (currently placeholder)
- Public API — listed under `/api-docs`; needs actual implementation
- Airtable integration — base/table config wired to `send-to-airtable` edge function
- Twilio integration — E.164 phone validation + send-to-twilio wiring
- Facebook OAuth — after DBA
- Apple OAuth — after Apple dev account
- Light mode — full light/dark theme toggle
- Search form simplification — current form is complex; simplify for non-technical users
- Enterprise plan — contact-based; no Stripe flow yet
- Mobile experience — current design is desktop-first; needs responsive QA pass

---

12. Deployment & Operations

Deploy process:

1. `npm run build` (must complete without errors — outputs to `build/`)
2. `git add -A && git commit -m "message" && git push origin main`
3. Vercel auto-deploys from `main` branch

Git note: Run git commands via cmd (not PowerShell) using `C:\Program Files\Git\bin\git.exe`.

Vercel project: `listing-bug` (team: `team_tvsjFBK7Xv5jiZhqdxEuhnUs`)

Edge function deploy:

```
npx supabase functions deploy <function-name> --project-ref ynqmisrlahjberhmlviz
```

Key config:

- Supabase project ID: `ynqmisrlahjberhmlviz`
- Supabase URL: `https://ynqmisrlahjberhmlviz.supabase.co`
- Vercel project ID: `prj_4D3zdZgjjaRmUkDwzgOh3RvxhvSK`

TDZ crash prevention: Never add a new top-level import to `App.tsx` for any component that itself imports from Supabase or shared modules. Use `lazy()` for all new components. If something must be synchronous, inline it as a function declaration after all imports.